AWS SAM and Serverless Applications

Explore the AWS Serverless Application Model (SAM) to define, build, and deploy scalable serverless applications. Understand SAM templates, CLI commands for local testing, and best development practices to improve efficiency and security in cloud-native applications.

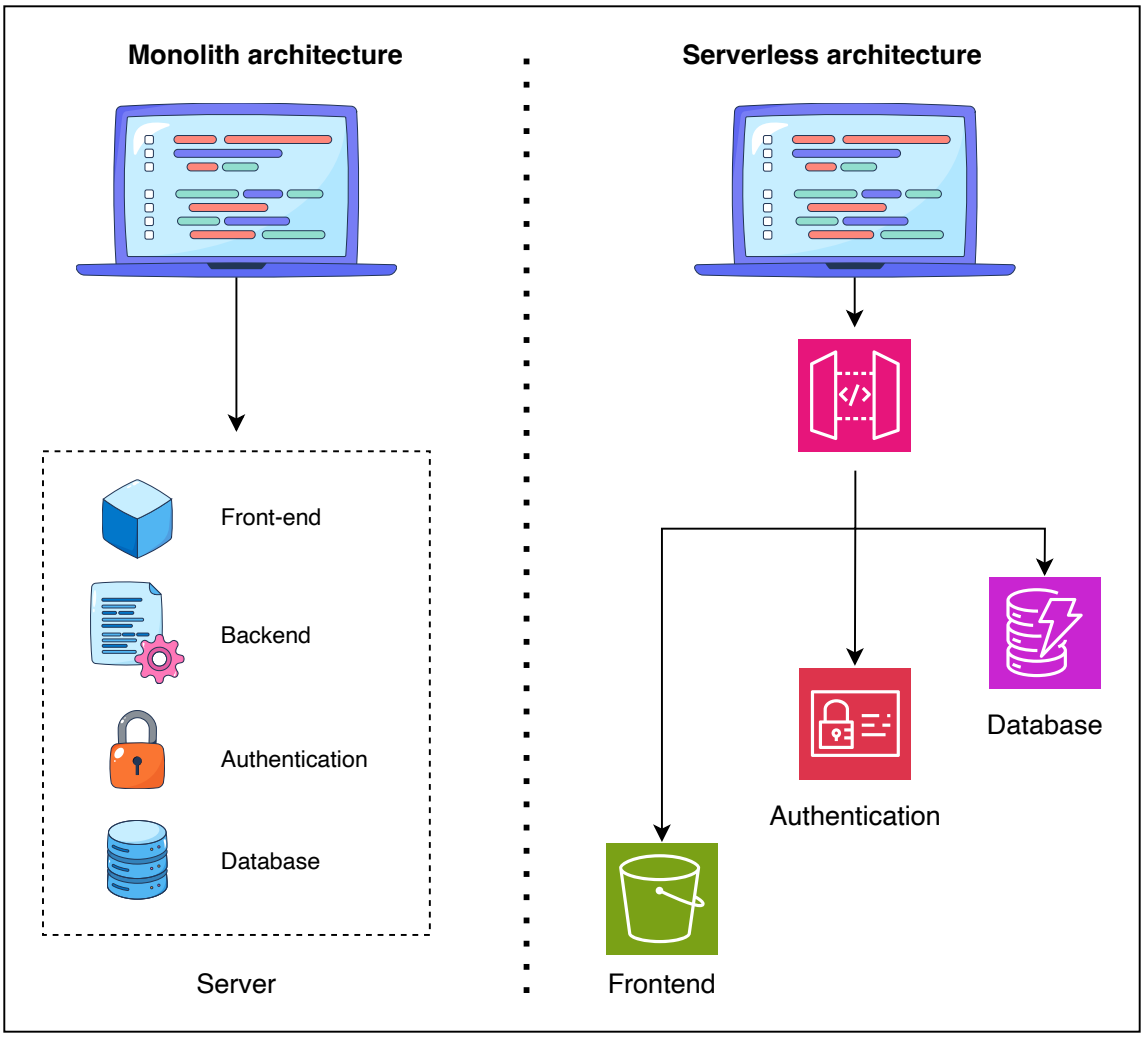
Traditionally, applications were designed on request/response models. The client would send requests to the backend, consisting of multiple classes to process the request and return responses. Since such applications are all bundled in one package, they are also called monolith applications.

With the arrival of cloud computing, developers deployed their monolith software applications on virtual machines on the cloud, such as an EC2 instance. They would install an operating system and the application code on these machines and configure a firewall to secure the application. If the demand spiked, they would spin up new EC2 instances, perform installations, and set up proxies to balance the traffic. In simple terms, they manually up- and down-scaled the compute resources.

However, concerns about simultaneous maintenance and development can be draining for the developer. This is where serverless computing comes in as the savior.

Serverless architecture is an architectural approach in which components, such as authentication, database, and business logic, are provided by serverless services. The entire application is designed to execute without the need to or configure any servers on the developer's end.

Ask



Monolith architecture vs. serverless architecture

Serverless Application Model (SAM)

AWS Serverless Application Model (AWS SAM) is an open-source framework specifically designed to simplify the development of serverless applications. It extends AWS CloudFormation and provides shorthand syntax to define functions, APIs, databases, and event source mappings.

SAM has two major components: the SAM template and the SAM CLI.

Structure of SAM templates

Ask

SAM templates are YAML-based configuration files that define the infrastructure and configuration of our serverless application. These templates are essentially

CloudFormation templates with simplified syntax for serverless resources.

- **AWSTemplateFormatVersion:** This section specifies the version of the AWS CloudFormation template format being used. While this is a required field, it does not usually change between projects. For example, most templates use `2010-09-09`, which indicates compatibility with the foundational version of CloudFormation. Though not critical to application behavior, it is required for SAM to parse the template correctly.
- **Transform:** `AWS::Serverless-2016-10-31` This field is specific to AWS SAM and indicates that the template should be transformed using the SAM macro. It enables shorthand syntax like `AWS::Serverless::Function` to be expanded into standard CloudFormation resources. Without this declaration, SAM resources will not be recognized. For instance, it tells AWS to interpret resources using SAM logic when deploying a SAM-based application.
- **Globals:** The `Globals` section lets us define properties that apply to all SAM resources of a given type, such as all functions or APIs. This eliminates repetition and helps ensure consistency. For example, if all our Lambda functions use `Python 3.11` and a 128 MB memory allocation, we can set these defaults once under `Globals`, rather than repeating them for every function.
- **Resources:** The `Resources` section is the heart of a SAM template, where all infrastructure components—like Lambda functions, APIs, and DynamoDB tables—are defined. Each resource uses a **logical ID**, a **Type**, and a **Properties** block.
    - The **Type** specifies the AWS or SAM resource. Each `AWS::Serverless::Function`, `AWS::Serverless::Api`, and `AWS::Serverless::Table` maps to AWS Lambda, API Gateway, and DynamoDB, respectively.
    - **Properties** defines its configuration, such as `Handler`, `Runtime`, `CodeUri`, and event sources under `Events`. For example, a

Ask

`HelloWorldFunction` might include a Python handler, a runtime, and an API trigger—all declared within this structured block.

- **Outputs:** The `Outputs` section defines values that can be exported for use in other stacks or displayed to the developer after deployment. For instance, we can output an API Gateway endpoint URL or Lambda function ARN, which is useful when integrating the deployed stack into other AWS services or for CI/CD pipelines.

Building a serverless app

Let's begin with a basic example of a Lambda function declared in a SAM template.

index.yaml YAML

```yaml
Resources:
  HelloWorldFunction:           # Logical name for the Lambda functio
    Type: AWS::Serverless::Function  # Declares this resource as a SA
    Properties:
      Handler: app.lambda_handler     # Specifies the entry point to
      Runtime: python3.11             # Defines the language runtime
      CodeUri: hello_world/           # Points to the directory conta
      MemorySize: 128                 # Allocates 128 MB of memory fo
      Timeout: 5                      # Sets the function timeout to
```

SAM template to deploy a Lambda function

This Lambda definition starts with `Type: AWS::Serverless::Function`, indicating that SAM will manage this function using its simplified syntax. Under `Properties`, the `Handler` field tells AWS which method to run—here, it's `lambda_handler` inside the `app.py` file. The `Runtime` is set to `python3.11`, which ensures compatibility with the syntax and dependencies in our code. `CodeUri`

Ask

designates the folder or location containing the source files. Performance and cost can be tuned using `MemorySize` and `Timeout`, which define the memory allocated and the maximum runtime duration. These settings are critical when optimizing serverless workloads.

Integrating API Gateway

To expose the Lambda function as an HTTP endpoint, SAM allows us to define an API Gateway integration directly within the function's events. We can embed the API inline or reference it separately using `AWS::Serverless::Api`.

index.yaml YAML

```yaml
Resources:
  HelloWorldApi:                      # Logical name for the API Gatev
    Type: AWS::Serverless::Api        # Declares an API Gateway using
    Properties:
      StageName: prod                 # Sets the deployment stage nam

  HelloWorldFunction:                 # Logical name for the Lambda fu
    Type: AWS::Serverless::Function   # Declares this resource as a S
    Properties:
      Handler: app.lambda_handler     # Entry point for Lambda execut
      Runtime: python3.11             # Runtime environment is Python
      CodeUri: hello_world/           # Path to the Lambda function c
      Events:
```

SAM template to deploy API Gateway with Lambda function

In this example, the `Events` property connects the function to a REST API. The `Type: Api` trigger specifies the API Gateway integration, while `Path` and `Method` define the route and HTTP verb. This setup makes it easy to prototype and deploy serverless backends without manually configuring API Gateway in the consol

Ask

Adding a DynamoDB Table

SAM also simplifies database integration by supporting `AWS::Serverless::SimpleTable` for quick setups or `AWS::DynamoDB::Table` for full configuration.

index.yaml YAML

```yaml
Resources:
  ItemsTable:                          # Logical name for the DynamoDB
    Type: AWS::Serverless::SimpleTable  # Uses SAM's shorthand to def
    Properties:
      PrimaryKey:
        Name: id                       # Name of the primary key attrib
        Type: String                   # Data type of the primary key a
```

SAM template to deploy a DynamoDB table

Using `AWS::Serverless::SimpleTable`, we can quickly define a DynamoDB table with a default `id` primary key. This is ideal for lightweight use cases. For more advanced scenarios—such as setting provisioned throughput, defining GSIs, or configuring TTL—we can use the full `AWS::DynamoDB::Table` resource.

With these building blocks—Lambda for compute, API Gateway for routing, and DynamoDB for storage—we can assemble a complete serverless application in a single SAM template. Each component is independently manageable, yet tightly integrated through the SAM abstraction, making deployment fast, repeatable, and scalable.

Ask

Test applications locally with SAM CLI

The AWS SAM CLI is a command-line tool that streamlines the development, testing, and deployment of serverless applications defined by AWS SAM templates. It lets us test our serverless applications locally by running them inside a Docker container that simulates the live environment. This allows us to invoke functions and test API endpoints on our machine, providing a rapid feedback loop to find and fix bugs before ever deploying to the cloud.

The CLI provides a set of commands that map directly to the life cycle of a serverless application.

Initialize a new application

This is the starting point for any new serverless project. We run the following command to initialise the serverless project:

main.sh Shell

```
sam init
```

This command creates a new serverless application from a pre-configured template. It generates a `template.yaml` file, starter Lambda function code folder structure, giving us a working "Hello World" application.

Ask

Build the application

Before testing or deploying, we need to build the application.

main.sh Shell

```
sam build
```

This command processes the `template.yaml` file, downloads any dependencies the functions need, for example, from `requirements.txt` for Python or `package.json` for Node.js, and places the prepared artifacts in a `.aws-sam/build` directory.

Test and debug locally

The SAM CLI allows us to test the application on the local machine before deploying it to the cloud, significantly speeding up the development cycle.

main.sh Shell

```
sam local invoke <FunctionLogicalId>
```

Ask

This command invokes a single Lambda function locally, allowing us to pass in an event payload and see the function's output. It's perfect for unit testing our function's logic.

main.sh Shell

```
sam local start-api
```

This command starts a local web server that emulates Amazon API Gateway. It lets us test the serverless APIs by sending HTTP requests to a local endpoint such as `http://127.0.0.1:3000`.

Deploy to the cloud

Once satisfied with the application, we can deploy it to the AWS account.

Ask

main.sh Shell

```
sam deploy --guided
```

This command packages the application, uploads the build artifacts to an S3 bucket, and deploys the application using AWS CloudFormation. The `--guided` flag initiates an interactive prompt that walks us through the deployment parameters for the first time, such as stack name and region, and saves them to a configuration file for future non-guided deployments.

Monitor and troubleshoot

After deployment, we can use the SAM CLI to monitor the live application.

main.sh Shell

```shell
sam logs --name <FunctionLogicalId> --stack-name educative-stack
```

Ask

This command fetches and streams logs for a specific Lambda function directly from Amazon CloudWatch Logs to the terminal, providing a real-time view of our function's activity.

main.sh Shell

```
sam traces --trace-id <TraceId>
```

If we have AWS X-Ray enabled, this command allows us to retrieve and view trace data for a specific request.

Apply best practices in SAM development

Here are some of the best practices for SAM development

- **Use `Globals` to avoid repeating configurations:** The `Globals` section in a SAM template allows us to define default properties for all functions, APIs, and other serverless resources. By doing this, us to ensure consistency and reduce redundancy, especially when multiple resources share common configurations like runtime or timeout settings.
- **Keep functions small and focused:** Serverless architecture benefits modularity. Each Lambda function should have a single responsibility to

Ask

improve maintainability, scalability, and testability. This aligns with the microservices approach and allows faster iterations during development.

- **Apply least privilege IAM roles per function:** Security is a core principle of AWS best practices. Ensure each function only has the permissions it absolutely needs to perform its task. Over-provisioned roles increase the risk of privilege escalation and unauthorized access to resources.
- **Use environment variables for configuration:** Environment variables are an effective way to decouple configuration settings from code. This practice allows us to reuse the same function code across multiple environments (like dev, test, and prod) while customizing behavior based on environment-specific variables.
- **Organize code for clarity:** Separate infrastructure from logic: Maintain a clean separation between infrastructure definition and application logic. Keeping our `template.yaml` and function code in distinct directories improves project readability and simplifies team collaboration.
- **Automate tests and deployments with CI/CD tools like AWS CodePipeline:** Continuous integration and continuous deployment pipelines help automate testing and deployment workflows. Using AWS CodePipeline with SAM ensures that every change is validated and deployed consistently, reducing manual effort and the risk of errors in production.

Conclusion

AWS SAM helps developers build scalable and maintainable serverless applications using declarative infrastructure-as-code. With a streamlined YAML syntax, integrated local testing, and simple deployment workflows, SAM enables faster development and cleaner environment management for modern cloud

Ask